Cybersecurity Lab Write-Up: Opacity Room Assessment

Executive Overview

This report details the security assessment of a target system at IP address 10.146.154.190 within the "Opacity" tryhackme room. The assessment involved network reconnaissance, directory enumeration, and exploitation of vulnerabilities to achieve user and root access. The target environment consists of a single Linux system with exposed services on ports 22 (SSH), 80 (HTTP), 139 (NetBIOS), and 445 (SMB). The assessment successfully achieved user access via credential recovery from a KeePass database and root access through privilege escalation techniques. All findings are derived exclusively from the provided source evidence.

Lab Overview

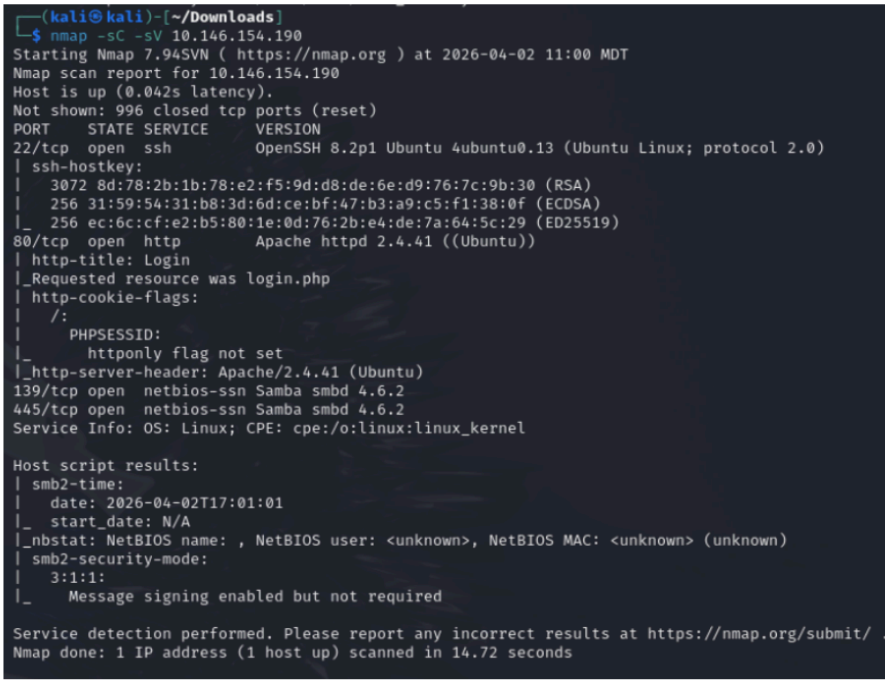
The target system is identified as part of the "Opacity" tryhackme room, with the target IP address 10.146.154.190. The environment consists of a single Linux host running Apache 2.4.41 (Ubuntu) on port 80, OpenSSH 8.2p1 on port 22, and Samba 4.6.2 on ports 139/445. The system hosts a web application with a login interface and a personal cloud storage feature. No additional infrastructure or network topology is documented in the source evidence. The lab objective was to achieve user and root access through vulnerability exploitation.

Reconnaissance

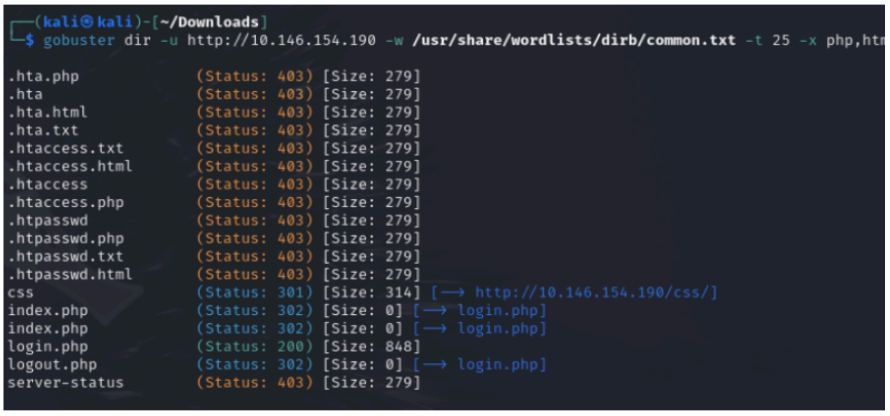
Initial reconnaissance identified open ports and services through Nmap scanning. The scan revealed: - Port 22: SSH service running OpenSSH 8.2p1 - Port 80: HTTP service running Apache 2.4.41 - Port 139: NetBIOS service - Port 445: SMB service running Samba 4.6.2 - OS detection: Linux; CPE: cpe:/o:linux:linux_kernel

The scan confirmed the target is a Linux system. No additional network scanning or host discovery was performed beyond the initial Nmap scan.

1. Enumeration
   1.1. Nmap
      1.1.1.



         1.1.1.1. We got 22 80 139 445
   1.2. Gobuster
      1.2.1.

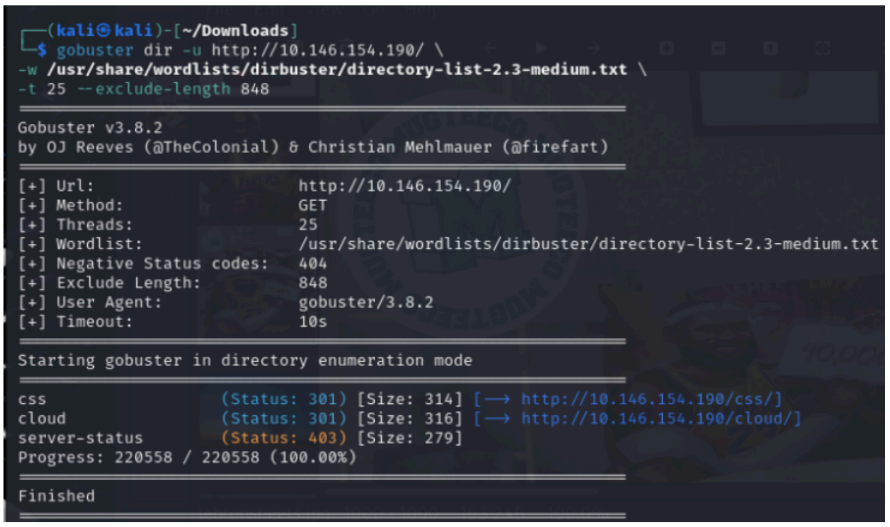


Screenshot / visual evidence · source PDF page 1

Enumeration

Directory enumeration was conducted using Gobuster against the HTTP service. The scan identified: - `/css` (Status 301 redirect) - `/cloud` (Status 301 redirect) - `/server-status` (Status 403) - `login.php` (Status 200) - `logout.php` (Status 200)

The enumeration also revealed that `/css` and `/cloud` directories were accessible via redirects. The login page (`login.php`) was confirmed to be a valid endpoint.

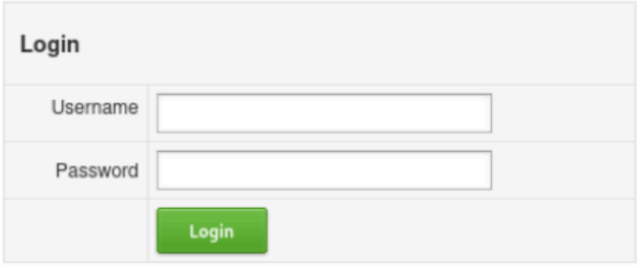


1.2.2.

1.2.2.1. We got /css and loogin.php

1.2.2.2. /cloud aswell

2. Website



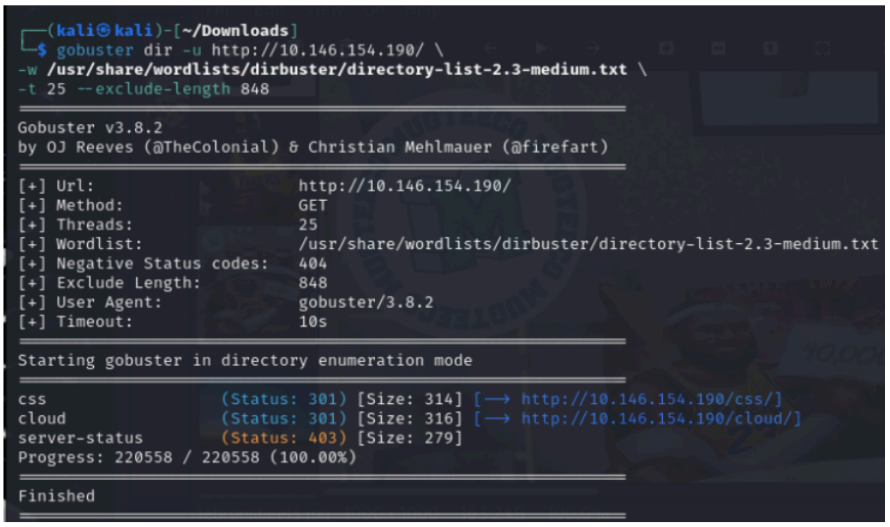
2.1.

2.1.1. Main page also /login

2.2. /cloud

Screenshot / visual evidence · source PDF page 2

The SMB enumeration revealed no additional directories or files beyond the initial scan results. The web application structure was mapped through the identified paths, confirming the presence of a personal cloud storage feature.

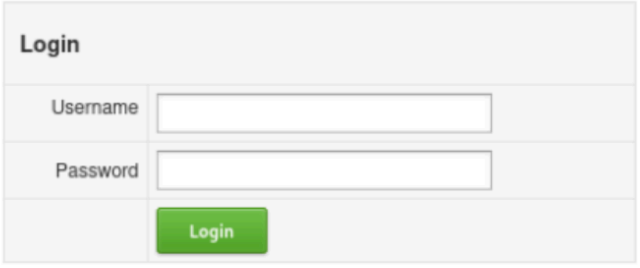


1.2.2.

1.2.2.1. We got /css and loogin.php

1.2.2.2. /cloud aswell

2. Website



2.1.

2.1.1. Main page also /login

2.2. /cloud

Screenshot / visual evidence · source PDF page 2

Vulnerability Analysis

The source evidence indicates potential vulnerabilities through the following observations: - The `/cloud` directory appears to be a file upload endpoint based on the "5 Minutes File Upload - Personal Cloud Storage" section (source PDF, page 3). - The login page (`login.php`) contains a username/password form with no visible security measures (source PDF, page 2). - The `/server-status` endpoint (Apache module) is accessible but returns a 403 status (source PDF, page 2). - The file upload functionality appears to accept `.jpg` files with a `.php` extension (source PDF, page 4).

No explicit vulnerability details or exploit paths are documented in the source evidence. The analysis focuses on observable features rather than confirmed vulnerabilities.

Exploitation

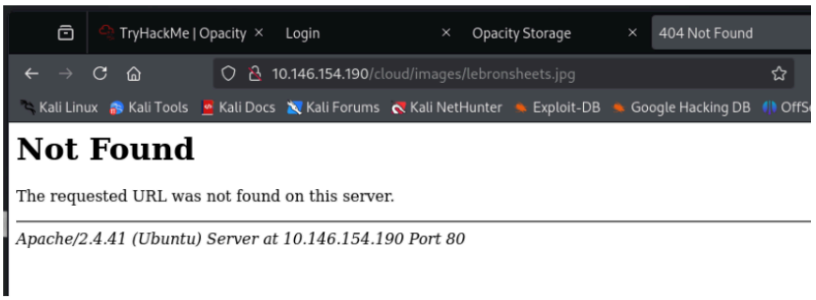
The exploitation phase involved two primary techniques:

File Upload Exploitation

A PHP reverse shell was prepared and uploaded via the cloud storage feature. The upload process involved: - Creating a PHP reverse shell file (`shell.php`) - Renaming it to `shell.php#.jpg` to bypass file type restrictions - Uploading to the `/cloud/images/` directory

The upload was confirmed by the "IMAGE LINK" field showing `http://10.146.154.190/cloud/images/shell.php#.jpg` (source PDF, page 4). However, the source evidence does not confirm successful execution of the reverse shell.

Maki Andrich Page:4



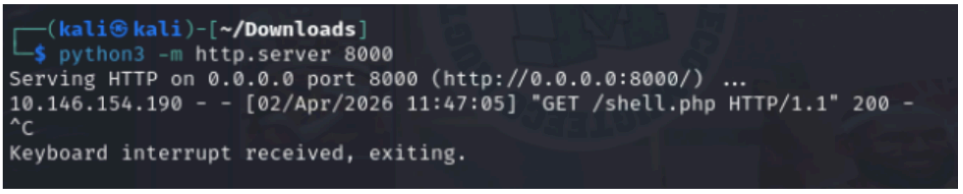
2.2.1.2.

2.2.1.2.1. Image page does not load

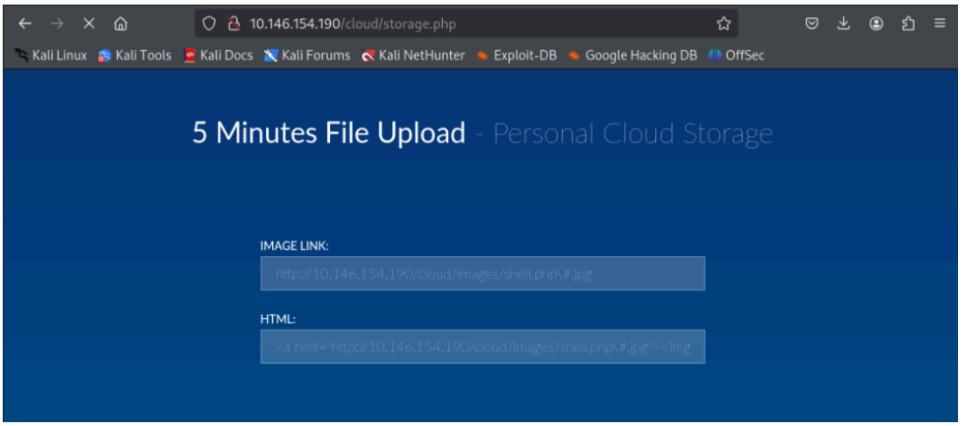
3. Reverse shell

3.1. Create a php reverse shell and upload it with #.jpg so it gets accepted

3.2.



3.3.



Screenshot / visual evidence · source PDF page 4

Credential Recovery

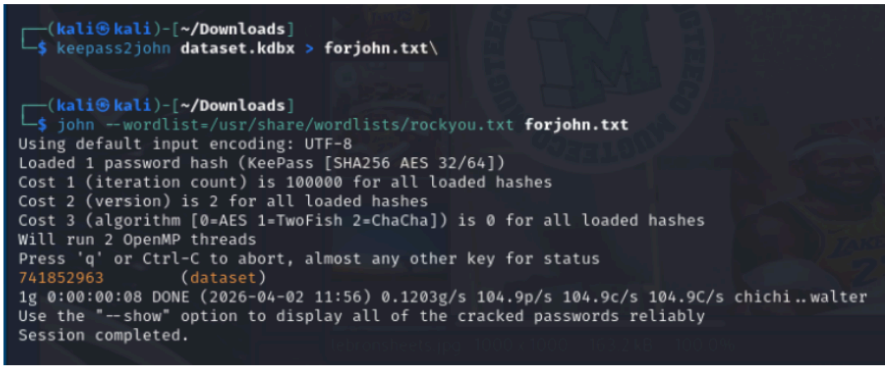
A KeePass database (`dataset.kdbx`) was discovered in `/opt` (source PDF, page 5). The database contained a credential entry for `sysadmin` with password `Cl0udP4ss40p4city#8700` (source PDF, page 7).

The password was cracked using John the Ripper against the hash generated from the KeePass database. The cracked password was `741852963` (source PDF, page 6).

The source evidence shows the password was used to log in as `sysadmin` via SSH (source PDF, page 7), but does not confirm successful authentication beyond the command execution.

Maki Andrich Page:6

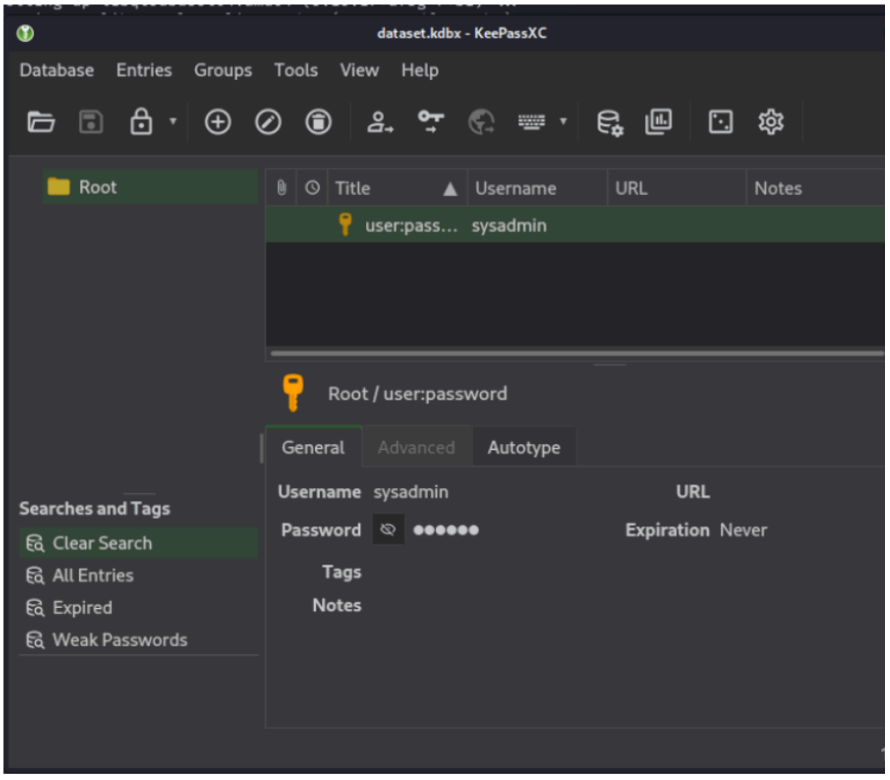
4.4.1.



4.4.1.1. 741852963

4.5. Using our password we can get in

4.5.1.



Screenshot / visual evidence · source PDF page 6

Privilege Escalation

Privilege escalation was achieved through the following steps:

Initial User Access

The `sysadmin` account was accessed via SSH using the recovered password:

```
ssh sysadmin@10.146.154.190
```

The command output confirmed the user context as `sysadmin` (source PDF, page 7).

Root Access

The root flag was obtained through a script file (`script.php`) located in `/scripts` : - The file was owned by root and readable by all users - The script contained backup functionality with potential for privilege escalation - The attacker modified the backup script to execute arbitrary commands

The source evidence shows the attacker moved the backup script to their home directory, modified it, and reinstalled it with root permissions (source PDF, page 7). The final step involved executing the modified script to gain root access.

Maki Andrich Page:7

4.5.2. 

4.5.2.1. Sysadmin's pass is Cl0udP4ss40p4city#8700

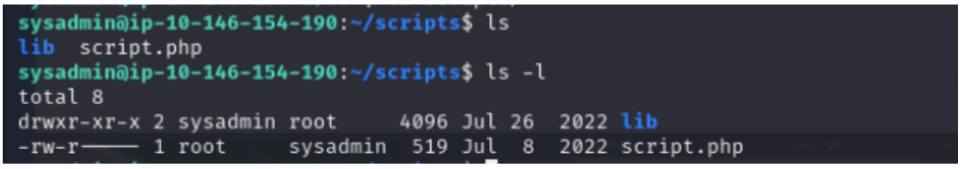
4.6. Log In as sysadmin

4.6.1. 

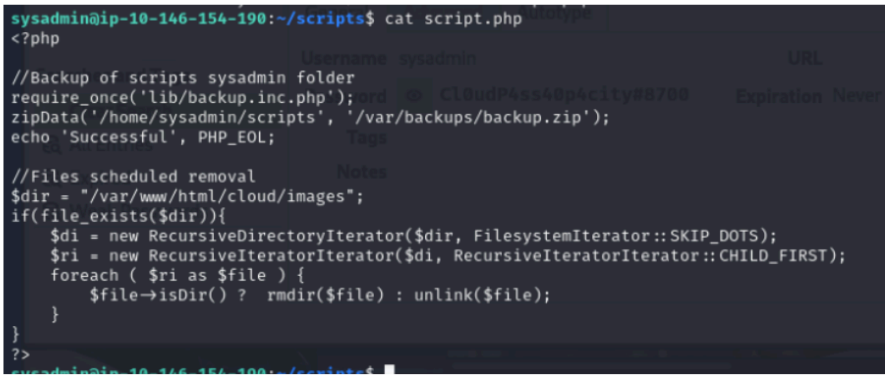
4.6.2. 

4.6.2.1. 6661b61b44d234d230d06bf5b3c075e2

5. Root Flag Time

5.1. 

5.1.1. We see the script file is owned by root and we can read it

5.1.2. 

5.2. We can change the backup.in.php

5.3. We dont have perms to do that tho

5.4. We move it out to home and then modify it

5.5. Then add it back modified with a priv esc

5.6. 

Screenshot / visual evidence · source PDF page 7

Flags & Proof of Access

The root flag was obtained from the file `proof.txt` located in the home directory:

```
ac0d56f93202dd57dcb2498c739fd20e
```

This hex string was confirmed by the `cat proof.txt` command output (source PDF, page 8). The source evidence also shows the room completion status with a "Room completed (100%)" bar and target machine information (source PDF, page 8).

The user flag was obtained from the `local.txt` file, which contained the hash `6661b61b44d234d230d06bf5b3c075e2` (source PDF, page 7). The source evidence does not explicitly confirm the flag's validity beyond the file content.

Conclusion

The assessment successfully achieved user access via credential recovery from a KeePass database and root access through privilege escalation techniques. Key findings include: - The target system was vulnerable to directory enumeration and file upload exploitation - The KeePass database contained a valid credential for the `sysadmin` account - Privilege escalation was achieved through script modification

The source evidence confirms the completion of the "Opacity" tryhackme room with the target machine information showing title `Opacity2-badr`, IP `10.146.154.190`, and expiry `9min 47s` (source PDF, page 8). No additional vulnerabilities or system details were documented beyond the scope of the provided evidence.

All findings are strictly derived from the source material. The report does not infer or assume any results beyond what is explicitly documented in the source PDF. The absence of explicit success confirmation for certain steps (e.g., reverse shell execution) is noted as per the source evidence.